# Progetto infrastruttura di Rete per Theta Spa

ArancinƏ Srls

# Sommario


1 SCOPO DEL PROGETTO ..... 3

2 OFFERTA COMMERCIALE ..... 3

3 PROGETTAZIONE INFRASTRUTTURA ..... 4

3.1 ANAGRAFICA DEI COMPONENTI ..... 4
3.2 TOPOLOGIA DI RETE ..... 4
3.3 INDIRIZZAMENTO IP ..... 5
3.4 CONFIGURAZIONE ROUTER ..... 5
3.5 CONFIGURAZIONE FIREWALL ..... 6
*3.5.1 Interfacce ASA ..... 6*
*3.5.2 NAT/PAT ..... 6*
*3.5.3 Access control list ..... 6*
*3.5.4 Routing ASA ..... 6*
3.6 ZONA DMZ ..... 7
3.7 NAS ..... 7
3.8 SONDE IPS/IDS ..... 7
3.9 CONNETTIVITÀ INTERNET – NOTA TECNICA ..... 8
3.10 RIEPILOGO STATO CONFIGURAZIONE ..... 8

4 CONFIGURAZIONE SOFTWARE ..... 9

4.1 CONFIGURAZIONE PFSENSE E KALI ..... 9
4.2 CONFIGURAZIONE METASPLOITABLE ..... 10
4.3 TEST DI RAGGIUNGIBILITÀ ..... 10
*4.3.1 Kali Linux → Metasploitable ..... 11*
*4.3.2 Metasploitable → Kali Linux ..... 11*

5 CONFIGURAZIONE PYTHON ..... 12

5.1 INTRODUZIONE ..... 12
5.2 VERIFICA DEI VERBI ..... 12
*5.2.1 Il codice in dettaglio ..... 13*
*5.2.2 Risultati del test ..... 15*
*5.2.3 Analisi ..... 16*
*5.2.4 Consigli di sicurezza il metodo TRACE ..... 16*
5.3 SCANSIONE DELLE PORTE ..... 17
*5.3.1 Il codice nel dettaglio ..... 18*
*5.3.2 Risultati del test ..... 21*
*5.3.3 Analisi ..... 21*
*5.3.4 Consigli di sicurezza, TELNET, FTP, http ..... 21*

6 CONCLUSIONI ..... 22

# 1 Scopo del progetto

Lo scopo del progetto è la realizzazione di una infrastruttura di rete ex novo per l'azienda Theta SpA.

La nuova infrastruttura si pone come obbiettivi il collegamento di 120 PC distribuiti su 6 piani dello stabile, un server DVWA, DHCP server, NAS server. La stessa dovrà essere protetta da Firewall, 3 IPS, 3 IDS ed avere connettività verso internet.

Il cablaggio dello stabile, l'approvvigionamento nonché il montaggio delle macchine richieste dalla progettazione e la messa insicurezza dei locali tecnici in conformità alle normative vigenti è a cura del cliente.

Tutti i dettagli, e l'offerta economica saranno a seguire nel documento.

# 2 Offerta commerciale

Dati i requisiti espressi da Theta SpA, di seguito l'offerta economica, divisa per area di competenza,

| | | | |
|---|---|---|---|
| Core e CED | Installazione DHCP Server<br>Configurazione DHCP Server<br>Configurazione VLAN<br>Installazione DVWA Server<br>Configurazione DVWA Server<br>Installazione NAS<br>Configurazione NAS | Totale | 3.000,00 € |
| Assistenza Core e CED | Assistenza commistioning<br>Assistenza migrazione | Totale | 1.120,00 € |
| Prerifiche HW | Commistioning Switch<br>Configurazione Switch | Totale | 2.016,00 € |
| Project Manag. | Quota Gestione Commessa<br>Coordinamento attività, sopralluoghi, SAL<br>Supporto progettazione | Totale | 2.960,00 € |
| Extra | Extra (riunioni, modifiche, integrazioni) | Totale | 2.000,00 € |
| Totale arrotondato | | 11.000,00 € | + IVA |

# 3 Progettazione Infrastruttura

La progettazione si è basata sui requisiti forniti da cliente, che includono:

- Cybersecurity compliance
- Distribuzione su 6 piani
- 120 host collegati via cavo (6 per piano)
- Connettività internet
- Configurazione NAS
- Configurazione Web Server

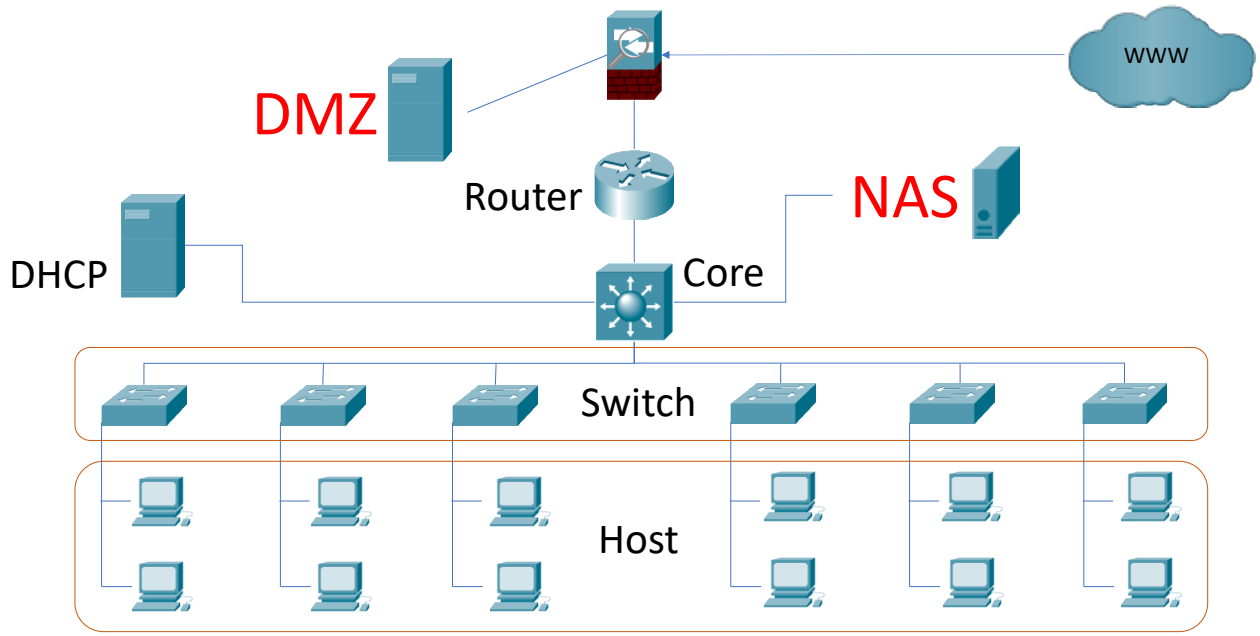
## 3.1 Anagrafica dei componenti

Il cliente si impegna a fornire tutto il cablaggio strutturale ove necessario e le componentistiche hardware e le licenze software necessarie all'implementazione richiesta. Nello specifico:

n1 Router Cisco
n1 Firewall Cisco
n1 Core switch Cisco
n6 Access switches Cisco
n1 Server NAS
n1 Server DVWA
n1 Connettività verso ISP
n120 PC generici

## 3.2 Topologia di Rete

La topologia di rete verrà così strutturata:



## 3.3 Indirizzamento IP

Per ottimizzare la rete abbiamo deciso di proporre una segmentazione per piano

| VLAN | Subnet | DHCP | Area / Note |
|---|---|---|---|
| VLAN 10 | 10.1.10.0/24 | ON (10.1.10.10→254) | Piano Terra – Access |
| VLAN 20 | 10.1.20.0/24 | ON (10.1.20.10→254) | Piano Primo – Access |
| VLAN 30 | 10.1.30.0/24 | ON (10.1.30.10→254) | Piano Secondo – Access |
| VLAN 40 | 10.1.40.0/24 | ON (10.1.40.10→254) | Piano Terzo – Access |
| VLAN 50 | 10.1.50.0/24 | ON (10.1.50.10→254) | Piano Quarto – Access |
| VLAN 60 | 10.1.60.0/24 | ON (10.1.60.10→254) | Piano Quinto – Access |
| VLAN 100 | 10.1.100.0/24 | OFF – Statico | Rete Servizi (Router↔ASA) |
| VLAN 200 | 10.1.200.0/24 | OFF – Statico | DMZ – Vlan Servizio |
| VLAN 300 | 10.1.300.0/24 | OFF – Statico | Management (future impl.) |

Le prime 6 Vlan di accesso sono quelle che fisicamente collegano i 120 host presenti all'interno dell'azienda, questo garantisce possibilità di implementazioni future nel caso di aggiunta di devices all'interno della rete.

La Vlan 100 collega gli apparati che erogano i servizi all'interno della rete (server DHCP, NAS), la Vlan 200, sempre di servizio, collega il firewall al DMZ, ed infine la Vlan 300 di management per la configurazione da remoto degli apparati.

## 3.4 Configurazione Router

Il Router è configurato con subinterface su Fa9/0 per il routing inter-VLAN. L'interfaccia Fa8/0 funge da uplink verso l'ASA con IP 10.1.100.1.

| Interfaccia | IP | Encapsulation | Ruolo |
|---|---|---|---|
| Fa9/0.10 | 10.1.10.1/24 | encap dot1Q 10 | Gateway VLAN 10 |
| Fa9/0.20 | 10.1.20.1/24 | encap dot1Q 20 | Gateway VLAN 20 |
| Fa9/0.30 | 10.1.30.1/24 | encap dot1Q 30 | Gateway VLAN 30 |
| Fa9/0.40 | 10.1.40.1/24 | encap dot1Q 40 | Gateway VLAN 40 |
| Fa9/0.50 | 10.1.50.1/24 | encap dot1Q 50 | Gateway VLAN 50 |
| Fa9/0.60 | 10.1.60.1/24 | encap dot1Q 60 | Gateway VLAN 60 |
| Fa8/0 | 10.1.100.1/24 | – | Uplink verso ASA (Inside) |

*Default route configurata: ip route 0.0.0.0 0.0.0.0 10.1.100.2 (verso ASA).*

## 3.5 Configurazione Firewall

### 3.5.1 Interfacce ASA

| Interfaccia | IP | Security Level | Ruolo |
|---|---|---|---|
| Vlan1 (inside) | 10.1.100.2/24 | 100 | Rete interna – uplink Router |
| Vlan2 (outside) | 200.0.0.1/24 | 0 | Connessione Internet (vedi nota) |
| Vlan3 (dmz) | 10.1.200.1/24 | 50 | Zona DMZ – Web Server |

*Nota: l'interfaccia outside (Vlan2) è stata temporaneamente disattivata (no nameif) per aggirare il limite della licenza base ASA 5505 in Packet Tracer che impedisce di usare più di 2 interfacce con forward attivo. In ambiente reale con licenza Security Plus questa limitazione non sussiste.*

### 3.5.2 NAT/PAT

Configurato PAT dinamico (overload) con Auto NAT tramite object network INSIDE_NET per la traduzione del traffico dalla rete interna verso l'esterno:

object network INSIDE_NET
subnet 10.1.0.0 255.255.0.0
nat (inside,outside) dynamic interface

### 3.5.3 Access control list

| ACL | Regola | Descrizione |
|---|---|---|
| INSIDE_OUT line 1 | permit ip any any | Traffico generico inside→outside |
| INSIDE_OUT line 2 | permit ip 10.1.0.0/16 any | Traffico rete interna verso qualsiasi dest. |
| INSIDE_OUT line 3 | permit ip 10.1.0.0/16 10.1.200.0/24 | Traffico inside→DMZ |
| DMZ_IN line 1 | permit icmp any any | ICMP da DMZ (risposte ping) |
| DMZ_IN line 2 | deny ip any 10.1.0.0/16 | Blocco DMZ→Inside (sicurezza) |
| DMZ_IN line 3 | permit ip any any | Altro traffico DMZ permesso |

*INSIDE_OUT applicata in interface inside. DMZ_IN applicata in interface dmz.*

### 3.5.4 Routing ASA

Aggiunta route statica per raggiungere le reti interne delle VLAN di piano:

```
route inside 10.1.0.0 255.255.0.0 10.1.100.1
```

## 3.6 Zona DMZ

Il server DMZ (Web Server simulato in PT) è posizionato nella rete 10.1.200.0/24, protetto dall'ASA. La comunicazione dalla rete interna verso la DMZ è permessa e verificata con successo. Il traffico dalla DMZ verso la rete interna è esplicitamente bloccato dalla ACL DMZ_IN (deny ip any 10.1.0.0/16).

| Dispositivo | IP | Subnet Mask | Gateway |
|---|---|---|---|
| Server DMZ (Web) | 10.1.200.2 | 255.255.255.0 | 10.1.200.1 (ASA DMZ) |

*Test eseguito: ping da PC VLAN 10→60 verso 10.1.200.2 — esito positivo (3/4 pacchetti, il primo timeout è atteso per ARP resolution).*

## 3.7 NAS

Il server NAS è stato posizionato in VLAN 10 (Piano Terra), collegato al Core Switch tramite switch dedicato IDS_IPS_SW0_Core. Accessibile da tutti i piani tramite routing inter-VLAN già operativo. Servizi attivi: HTTP, FTP (simulazione accesso file).

| Dispositivo | IP | Subnet Mask | Gateway |
|---|---|---|---|
| NAS_Piano_Terra | 10.1.10.5 | 255.255.255.0 | 10.1.10.1 (Router VLAN 10) |

*Test eseguito: ping da PC VLAN 60 verso 10.1.10.5 — esito positivo. Il NAS è raggiungibile da qualsiasi host della rete aziendale.*

## 3.8 Sonde IPS/IDS

Come indicato nel documento di progetto, sono state previste coppie di sonde IDS/IPS nelle seguenti posizioni:

| Dispositivo | IP | Posizione | Funzione |
|---|---|---|---|
| IDS_SW0_Core | 10.1.100.20 | Core Switch | Monitoraggio traffico aggregato |
| IPS_SW0_Core | 10.1.100.21 | Core Switch | Prevenzione intrusioni perimetro interno |
| IDS_NAS / IPS_NAS | TBD | Interno NAS | Monitoraggio accessi storage |
| IDS_DMZ / IPS_DMZ | TBD | Zona DMZ | Protezione da attacchi esterni |

*Nota: Cisco Packet Tracer non include dispositivi IDS/IPS nativi. Le sonde sono simulate tramite server generici posizionati nei punti strategici della topologia. In un ambiente reale con hardware dedicato (es. Cisco Firepower, Snort) la configurazione sarebbe pienamente operativa.*

## 3.9 Connettività Internet – nota tecnica

La configurazione NAT/PAT verso l'esterno è stata implementata correttamente (translate_hits e untranslate_hits confermati tramite show nat). La connettività verso Internet non risulta testabile in Packet Tracer per due motivi:

- PT non ha accesso a internet reale — qualsiasi IP esterno non configurato nella topologia non risponde.
- L'ASA 5505 con licenza base in PT supporta massimo 2 interfacce attive con forward abilitato.

Avendo attivato inside e DMZ, l'interfaccia outside (Vlan2) è stata temporaneamente disattivata per consentire il test della DMZ.

## 3.10 Riepilogo stato configurazione

| Componente | Stato | Note |
|---|---|---|
| VLAN 10-60 (Access) | ✓ **Operativo** | DHCP funzionante, ping inter-VLAN ok |
| Core Switch Trunk | ✓ **Operativo** | Native VLAN 99, tutte le VLAN permesse |
| Router Inter-VLAN | ✓ **Operativo** | Subinterface Fa9/0.10→60 configurate |
| ASA Inside | ✓ **Operativo** | 10.1.100.2, security-level 100 |
| ASA DMZ | ✓ **Operativo** | 10.1.200.1, security-level 50, forward ok |
| ASA Outside | ■ **Limitazione PT** | Disattivata per limite licenza base ASA 5505 |
| NAT/PAT | ✓ **Configurato** | translate_hits=17 confermati |
| ACL INSIDE_OUT | ✓ **Operativo** | hitcnt attivi, traffico inside→DMZ ok |
| ACL DMZ_IN | ✓ **Operativo** | ICMP permit, blocco DMZ→Inside attivo |
| Server DMZ | ✓ **Raggiungibile** | Ping da qualsiasi VLAN verso 10.1.200.2 ok |
| NAS Piano Terra | ✓ **Raggiungibile** | Ping da qualsiasi VLAN verso 10.1.10.5 ok |
| IDS/IPS | ■ **Simulati** | Server generici PT, non testabili funzionalmente |
| VLAN 300 Management | ■ **Pianificata** | Inserita nella lista VLAN, future implementazioni |

# 4 Configurazione software

Di seguito andremo ad analizzare le configurazioni sulle macchine implementate.

## 4.1 Configurazione PfSense e Kali

```
WAN (wan) -> em0 -> v4/DHCP4: 10.0.2.15/24
                    v6/DHCP6: fd17:625c:f037:2:a00:27ff:fe4b:1887/64
LAN (lan) -> em1 -> v4: 10.1.100.1/24
```

Nel laboratorio è stato utilizzato un firewall pfSense configurato con due interfacce di rete distinte, ciascuna con un ruolo specifico all'interno dell'architettura virtuale:

- Interfaccia WAN: collegata alla modalità NAT di VirtualBox, configurata per ottenere automaticamente un indirizzo IP tramite DHCP. Questa interfaccia consente al firewall di accedere alla rete esterna e di instradare il traffico verso Internet.
- Interfaccia LAN: collegata a una rete interna di VirtualBox, configurata con un indirizzo IP statico assegnato manualmente. Tale interfaccia rappresenta la rete locale gestita da pfSense e ospita i dispositivi interni del laboratorio.

Sulla rete LAN è stato inoltre attivato il servizio DHCP di pfSense, definendo un intervallo di assegnazione dinamica compreso tra: 10.1.100.100 e 10.1.100.200

Questo range è stato utilizzato per fornire automaticamente indirizzi IP ai sistemi client presenti nella rete interna. La corretta funzionalità del servizio è stata verificata tramite una macchina virtuale Kali Linux, configurata per ottenere l'indirizzo IP in modalità DHCP. Al momento dell'avvio, Kali ha ricevuto un indirizzo appartenente al range definito, confermando il corretto funzionamento del server DHCP e la comunicazione tra pfSense e la rete interna. Di seguito, la dimostrazione dell'indirizzo ottenuto dalla macchina virtuali kali:

```
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.1.100.101  netmask 255.255.255.0  broadcast 10.1.100.255
        inet6 fe80::a00:27ff:fe23:335a  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:23:33:5a  txqueuelen 1000  (Ethernet)
        RX packets 80  bytes 12136 (11.8 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 67  bytes 7254 (7.0 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 8  bytes 480 (480.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 8  bytes 480 (480.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

## 4.2 Configurazione Metasploitable

All'interno della stessa infrastruttura virtuale è stata configurata anche una macchina **Metasploitable**. A differenza della macchina Kali, che ottiene l'indirizzo IP tramite il servizio DHCP di pfSense, Metasploitable è stata configurata con un **indirizzo IP statico**.

```
  GNU nano 2.0.7          File: /etc/network/interfaces

# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
auto eth0
iface eth0 inet static
address 10.1.100.10
netmask 255.255.255.0
gateway 10.1.100.1
```

L'interfaccia di rete della macchina è stata collegata alla medesima rete interna gestita da pfSense e condivisa con Kali Linux. In questo modo entrambe le macchine risultano appartenenti alla stessa sottorete e possono comunicare direttamente attraverso il router pfSense.

La configurazione statica dell'interfaccia di rete di Metasploitable include:

- Indirizzo IP assegnato manualmente all'interno della subnet LAN
- Netmask coerente con la configurazione della rete interna
- Gateway impostato sull'indirizzo LAN di pfSense, così da permettere l'instradamento del traffico verso gli altri segmenti di rete e verso l'esterno

Questa configurazione garantisce che Metasploitable sia correttamente integrata nella topologia di rete del laboratorio, permettendo la comunicazione con Kali Linux e con pfSense, e rendendola pienamente operativa per le attività di test e simulazione.

## 4.3 Test di raggiungibilità

Per accertare il corretto funzionamento della rete interna e la capacità delle due macchine virtuali di comunicare attraverso il router pfSense, è stata eseguita una procedura di verifica basata sull'utilizzo del comando *ping*. Tale test consente di valutare la raggiungibilità reciproca dei due host e di confermare la corretta configurazione degli indirizzi IP, del gateway e delle interfacce di rete.

### 4.3.1 Kali Linux → Metasploitable

```
└─$ ping 10.1.100.10
PING 10.1.100.10 (10.1.100.10) 56(84) bytes of data.
64 bytes from 10.1.100.10: icmp_seq=1 ttl=64 time=0.528 ms
64 bytes from 10.1.100.10: icmp_seq=2 ttl=64 time=0.780 ms
64 bytes from 10.1.100.10: icmp_seq=3 ttl=64 time=0.168 ms
^C
── 10.1.100.10 ping statistics ──
3 packets transmitted, 3 received, 0% packet loss, time 2037ms
rtt min/avg/max/mdev = 0.168/0.492/0.780/0.251 ms
```

La macchina Kali, configurata per ottenere automaticamente l'indirizzo IP tramite il server DHCP di pfSense, ha inviato una serie di pacchetti ICMP verso l'indirizzo IP statico assegnato alla macchina Metasploitable. Il test ha restituito risposte regolari, con tempi di latenza molto bassi e assenza di perdita di pacchetti, confermando la corretta raggiungibilità dell'host di destinazione.

### 4.3.2 Metasploitable → Kali Linux

```
PING 10.1.100.101 (10.1.100.101) 56(84) bytes of data.
64 bytes from 10.1.100.101: icmp_seq=1 ttl=64 time=0.559 ms
64 bytes from 10.1.100.101: icmp_seq=2 ttl=64 time=0.763 ms
64 bytes from 10.1.100.101: icmp_seq=3 ttl=64 time=0.754 ms
64 bytes from 10.1.100.101: icmp_seq=4 ttl=64 time=0.186 ms
64 bytes from 10.1.100.101: icmp_seq=5 ttl=64 time=0.752 ms
64 bytes from 10.1.100.101: icmp_seq=6 ttl=64 time=0.810 ms
64 bytes from 10.1.100.101: icmp_seq=7 ttl=64 time=0.579 ms

--- 10.1.100.101 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 5997ms
rtt min/avg/max/mdev = 0.186/0.629/0.810/0.201 ms
```

Successivamente, è stato effettuato il test inverso, inviando pacchetti ICMP dalla macchina Metasploitable verso l'indirizzo IP assegnato dinamicamente a Kali. Anche in questo caso il test ha avuto esito positivo, con ricezione di tutte le risposte e valori di latenza coerenti con una rete interna virtualizzata.
L'esito positivo di entrambi i test dimostra che:

- le due macchine si trovano correttamente nella stessa sottorete gestita da pfSense
- il routing interno è configurato in modo appropriato
- la configurazione degli indirizzi IP, del gateway e della subnet è coerente e funzionale

Questa verifica conferma la piena operatività dell'infrastruttura di rete predisposta per il laboratorio.

# 5 Configurazione Python

## 5.1 Introduzione

L'obiettivo del progetto è stato verificare il corretto funzionamento e il livello di sicurezza dell'infrastruttura di rete della compagnia Theta attraverso attività di analisi e testing su una macchina Metasploitable 2, utilizzata come Web Server di laboratorio.

L'operazione di **analisi** ha riguardato:

1. **Verifica dei verbi** (http) abilitati sul servizio web.
2. **Scansione delle porte** per identificare eventuali servizi esposti dalla macchina

Gli strumenti utilizzati sono stati sviluppati in linguaggio Python.

## 5.2 Verifica dei verbi

L'obiettivo del test è stato quello di testare e identificare i verbi http abilitati sul servizio web della macchina Metasploitable 2, in particolare sul path relativo a phpMyAdmin.

Lo strumento sviluppato in Python è configurato come segue:

- Riceve in input URL e PATH
- Invia richieste http utilizzando diversi verbi ( GET, POST, PUT, DELETE, OPTIONS, HEAD, PATH, TRACE)
- Analisi dello status dei verbi restituiti dal server
- Determinazione del metodo (Abilitato, Disabilitato)

Le librerie utilizzate sono state:

- Requests
- Datetime
- ZoneInfo

### 5.2.1 Il codice in dettaglio

Il codice sviluppato per il testing dei verbi http è il seguente

```
1  import requests
2  from datetime import datetime
3  from zoneinfo import ZoneInfo
4
5  def check_http_verbs(url, path):
6      full_url = url.rstrip("/") + path
7      verbs = ["GET", "POST", "PUT", "DELETE",
8               "OPTIONS", "HEAD", "PATCH", "TRACE"]
9
10     print(f"\n{'=' * 55}")
11     print(f"  TEST HTTP VERBS")
12     print(f"  URL: {full_url}")
13     print(f"  Data: {datetime.now(ZoneInfo('Europe/Rome')).strftime('%d/%m/%Y %H:%M:%S')}")
14     print(f"{'=' * 55}")
15
16     abilitati = []
17     disabilitati = []
18
19     for verb in verbs:
20         try:
21             r = requests.request(verb, full_url,
22                                  timeout=3,
23                                  allow_redirects=False)
24             if r.status_code < 400:
25                 print(f"  ✅ {verb:10} → {r.status_code} ABILITATO")
26                 abilitati.append(verb)
27             else:
28                 print(f"  ❌ {verb:10} → {r.status_code} DISABILITATO")
29                 disabilitati.append(verb)
30         except requests.exceptions.ConnectionError:
31             print(f"  ⚠️  {verb:10} → CONNESSIONE RIFIUTATA")
32         except requests.exceptions.Timeout:
33             print(f"  ⚠️  {verb:10} → TIMEOUT")
34         except Exception as e:
35             print(f"  ⚠️  {verb:10} → ERRORE: {e}")
36
37     print(f"\n{'=' * 55}")
38     print(f"  RIEPILOGO:")
39     print(f"  Verbi abilitati:    {abilitati}")
40     print(f"  Verbi disabilitati: {disabilitati}")
41     print(f"{'=' * 55}\n")
42
43 if __name__ == "__main__":
44     url  = input("URL base (es. http://172.16.0.2/): ").strip()
45     path = input("Path (es. /phpmyadmin/ ): ").strip()
46     check_http_verbs(url, path)
47
```

RIGA 1:

Nel dettaglio è stato progettato partendo dalla libreria requests. Questa libreria serve per fare le "richieste" http aprendo il sito web desiderato.

RIGA 2 e 3:

È stato importata la libreria datetime e zoneinfo per poter riportare la data in giorni, mesi, anni e gestisce anche l'orario aggiustato alla zona di fuso orario di Roma.

RIGA 5

Si è creata e definita la funzione che riceve l'url e il path.

RIGA 6

Ci permette di rimuovere eventuali errori di battitura relativi all'url e al path.

RIGA 7 e 8:

Qui abbiamo definito la lista dei verbi http da testare.

RIGA 10 a 14:

Si è voluto formattare graficamente per l'utente il titolo, l'url analizzato e cosa fa il programma.

RIGA 16 e 17:

Salva i metodi abilitati e disabilitati.

RIGA 19 e 20:

Apertura del ciclo for, scorre tutti i metodi http e per ognuno di essi invia una richiesta.

RIGA 21 e 22

La parte centrale del programma, invia richieste dinamiche http per ogni verbo aspettando 3 secondi.

RIGA 23:

L'opzione allow_redirects=False impedisce al programma di seguire automaticamente eventuali redirect del server. Questo è utile durante i test di sicurezza perché permette di vedere direttamente la risposta reale del server senza essere reindirizzati ad altre pagine.

RIGA da 24 a 39:

Qui viene controllato lo status code della risposta HTTP ricevuta dal server. Se il codice HTTP restituito è minore di 400, il verbo viene considerato abilitato. Il programma stampa il metodo HTTP, il relativo status code e salva il verbo nella lista dei metodi abilitati. Se invece il server restituisce uno status code maggiore o uguale a 400, il metodo viene considerato disabilitato. Anche in questo caso il programma stampa il risultato e salva il verbo nella lista dei metodi disabilitati.

RIGA 30 e 31:

Gestione dell'errore di connessione. Se il server non è raggiungibile oppure la porta è chiusa, viene mostrato all'utente il messaggio "CONNESSIONE RIFIUTATA".

RIGA 32 e 33:

Gestione del timeout. Se il server impiega troppo tempo a rispondere, il programma interrompe la richiesta e mostra il messaggio "TIMEOUT".

RIGA 34 e 35:

Gestione generica degli errori. Qualsiasi altro errore viene catturato e mostrato all'utente insieme alla descrizione dell'eccezione.

RIGA 37 a 41:

Questa sezione stampa un riepilogo finale del test mostrando:

1. i verbi HTTP abilitati;

2. i verbi HTTP disabilitati;

3. una separazione grafica per migliorare la leggibilità del risultato finale.

RIGA 43:

La condizione: if __name__ == "__main__": serve per eseguire il programma solo quando il file Python viene avviato direttamente.

RIGA 44:

Viene richiesto all'utente di inserire l'URL base del server da analizzare.

RIGA 45:

Viene richiesto il path specifico della risorsa da testare.

RIGA 46:

Infine, viene richiamata la funzione check_http_verbs() passando come parametri l'URL e il path inseriti dall'utente, avviando così il test dei verbi HTTP.

### 5.2.2 Risultati del test

Il programma ha permesso di identificare correttamente i metodi HTTP abilitati e disabilitati sul server Metasploitable 2 all'indirizzo http://10.1.100.10/ e al path /phpmyadmin/.

Dai risultati ottenuti è emerso che:

- Il metodo "option" risulta abilitato
- Il metodo "trace" risulta abilitato
- I metodi Put, Delete e Patch risultano bloccati dal server
- I metodi Get, Post, Head restituiscono errore sul path analizzato

```
┌──(simone㉿simone)-[~]
└─$ /usr/bin/python /home/simone/Documenti/HTTP_controllo_verbi.py
URL base (es. http://172.16.0.2/): http://10.1.100.10/
Path (es. /phpmyadmin/ ): /phpmyadmin/

=======================================================
  TEST HTTP VERBS
  URL: http://10.1.100.10/phpmyadmin/
  Data: 29/04/2026 11:47:53
=======================================================
  ❌ GET        → 404 DISABILITATO
  ❌ POST       → 404 DISABILITATO
  ❌ PUT        → 405 DISABILITATO
  ❌ DELETE     → 405 DISABILITATO
  ✅ OPTIONS    → 200 ABILITATO
  ❌ HEAD       → 404 DISABILITATO
  ❌ PATCH      → 405 DISABILITATO
  ✅ TRACE      → 200 ABILITATO

=======================================================
  RIEPILOGO:
  Verbi abilitati:    ['OPTIONS', 'TRACE']
  Verbi disabilitati: ['GET', 'POST', 'PUT', 'DELETE', 'HEAD', 'PATCH']
=======================================================
```

### 5.2.3 Analisi

Dal test emerge che:

**OPTIONS**

Il metodo OPTIONS risulta abilitato (200 OK).

Questo comportamento è normale poiché il server consente ai client di interrogare i metodi supportati.

**PUT, DELETE e PATCH**

I metodi PUT, DELETE e PATCH restituiscono 405 Method Not Allowed.

Ciò indica che:

- il server riceve correttamente la richiesta;
- il metodo è riconosciuto ma non è consentito sul path richiesto.

**GET, POST e HEAD**

I metodi GET, POST e HEAD restituiscono 404 Not Found.

Questo risultato indica che:

- il path specificato non contiene una risorsa accessibile tramite tali metodi.

### 5.2.4 Consigli di sicurezza il metodo TRACE

La squadra incaricata della sicurezza **consiglia a Theta di disabilitare il metodo http TRACE** sul web server, soprattutto in ambienti di produzione, al fine di ridurre il rischio di attacchi legati alla divulgazione di informazioni sensibili.

In particolare, il metodo **TRACE** potrebbe consentire a un attaccante di:

- intercettare o riflettere header HTTP contenenti informazioni riservate;
- raccogliere dettagli sulla configurazione del server;
- sfruttare vulnerabilità di sistema

Per mitigare tali rischi **si raccomanda di**:

- disabilitare il metodo TRACE nella configurazione del web server Apache;
- limitare i metodi HTTP esclusivamente a quelli necessari;
- monitorare i log del server per individuare richieste anomale;

implementare regole firewall e controlli di sicurezza aggiuntivi.

## 5.3 Scansione delle porte

La seconda fase del progetto Python ha riguardato la scansione delle porte TCP della macchina Metasploitable 2, con l'obiettivo di identificare i servizi attivi e verificare quali porte risultassero aperte o chiuse.

Questa attività è fondamentale nell'ambito della sicurezza informatica, poiché permette di comprendere quali servizi siano esposti in rete e quali possano rappresentare potenziali superfici di attacco.

Per svolgere il test è stato sviluppato uno scanner personalizzato in linguaggio Python.

Il programma realizzato riceve in input un indirizzo IP e procede ad effettuare una scansione TCP su una serie di porte predefinite, tentando una connessione verso ciascuna di esse.

Per ogni porta analizzata, il programma tenta di stabilire una connessione con il dispositivo target: se la connessione viene accettata, la porta viene considerata aperta; se la connessione viene rifiutata o non riceve risposta, la porta viene considerata chiusa. Attraverso questa procedura è stato possibile individuare i principali servizi esposti dalla macchina Metasploitable 2.

Il programma dunque funziona nel seguente modo:

1. riceve in input un indirizzo ip
2. effettua una scansione (in questo caso TCP)
3. determina lo stato delle porte
4. ci ritorna l'output delle porte (aperte o chiuse)

La libreria utilizzata è:

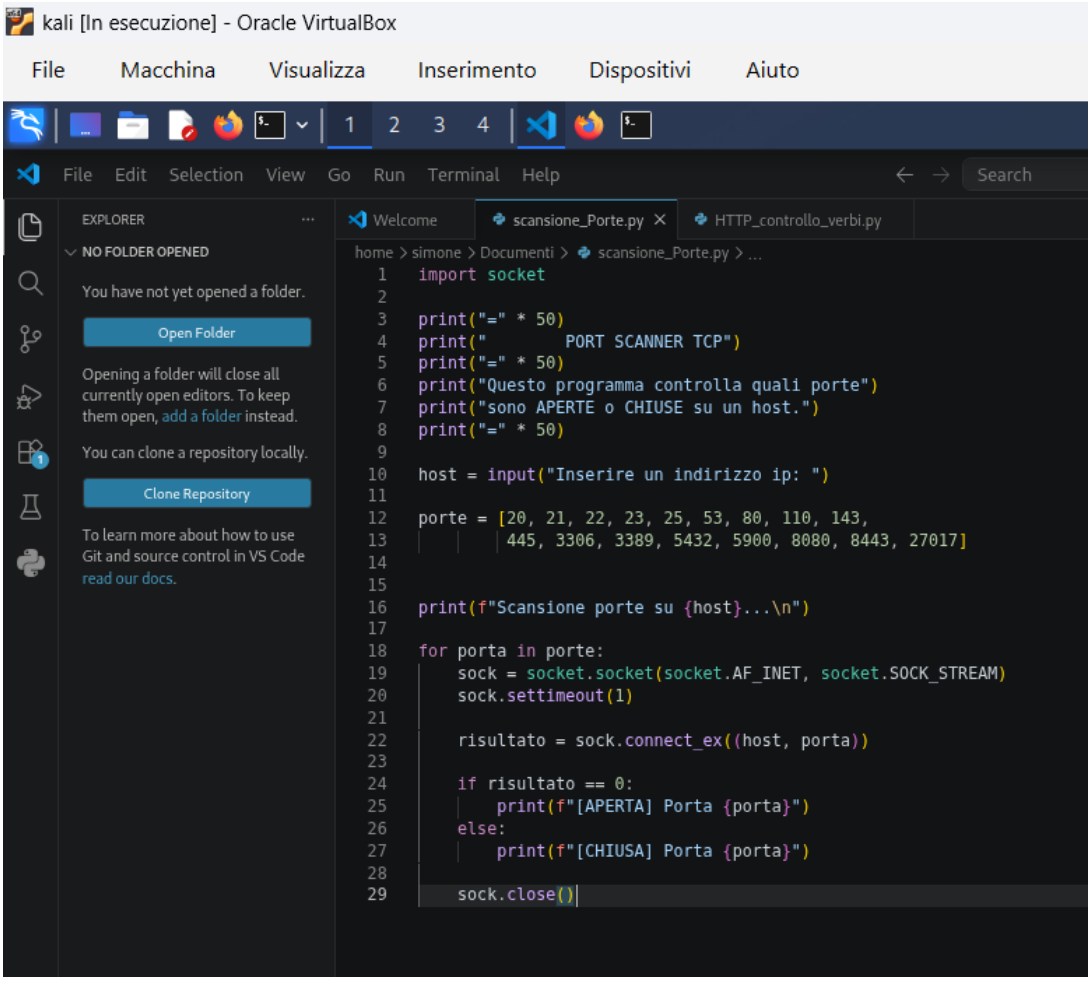
- Socket

La struttura precedentemente enunciata ci ha permesso in particolare di scansionare tra un pool di porte preimpostato quali di queste fossero "aperte" o "chiuse" all'indirizzo IP 10.1.100.10 ( Metasploitable 2).

Il pool di porte è stato scelto perché comprende alcune delle porte TCP più comuni e maggiormente utilizzate dai principali servizi di rete. L'obiettivo è stato quello di verificare la presenza di servizi frequentemente utilizzati in ambito server, amministrazione remota, database, servizi web e condivisione di rete, in modo da ottenere una panoramica generale della superficie di attacco esposta dalla macchina target.

### 5.3.1 Il codice nel dettaglio

Il codice sviluppato per la scansione delle porte è il seguente




RIGA 1:

Abbiamo importato la libreria socket, fondamentale per la comunicazione in rete di Python. Questa libreria consente appunto di stabilire una connessione permettendo al programma di verificare se una determinata porta di rete risulti aperta o chiusa su un host.

RIGA 3:

Per rendere l'esperienza dell'utente migliore si è voluto implementare una linea grafica composta da 50 caratteri = per migliorare la leggibilità dell'interfaccia del programma.

RIGA 4:

Stampa il titolo del programma, indicando all'utente che si tratta di uno scanner di porte TCP.

RIGA 5:

Stampa una seconda linea grafica di separazione.

RIGA 6 e 7:

Sempre in ottica di rendere migliore e più comprensibile all'utente queste righe mostrano una breve descrizione del funzionamento del programma, spiegando all'utente che il tool verifica lo stato delle porte di rete di un determinato host.

RIGA 8:

Viene stampata un'ulteriore linea grafica per separare l'introduzione dall'input utente.

RIGA 10:

Il programma richiede all'utente di inserire un indirizzo IP. L'indirizzo inserito viene salvato nella variabile host.

RIGA 12 e 13:

Qui viene definita una lista di porte TCP da analizzare. Ogni numero corrisponde a un servizio di rete comunemente utilizzato.

RIGA 16:

Il programma comunica all'utente l'inizio della scansione, mostrando l'indirizzo IP analizzato.

RIGA 18:

Apertura del ciclo for. Il programma scorre tutte le porte presenti nella lista e ripete il test per ognuna di esse.

RIGA 19:

Viene creato un socket TCP/IP.

Nel dettaglio:

- AF_INET indica l'utilizzo di indirizzi IPv4;
- SOCK_STREAM indica l'utilizzo del protocollo TCP.

Questo socket verrà utilizzato per tentare la connessione alla porta target.

RIGA 20:

Imposta un timeout di 1 secondo.

Questo significa che il programma attenderà massimo 1 secondo per ricevere una risposta dalla porta analizzata, evitando che la scansione diventi troppo lenta in caso di mancata risposta.

RIGA 22:

Questa rappresenta la parte centrale dello scanner.

La funzione connect_ex() tenta di stabilire una connessione TCP verso:

- l'host inserito dall'utente;
- la porta corrente del ciclo.

In particolare, la funzione connect_ex() restituisce:

- 0 se la connessione va a buon fine
- Un valore diverso da 0 se la connessione fallisce

RIGA 24:

Controlla se il valore restituito è uguale a 0.

Se la connessione è riuscita:

- la porta viene considerata aperta.

RIGA 25:

Il programma stampa il messaggio che indica la porta come aperta.

RIGA 26:

Se la connessione non riesce:

- il programma entra nel ramo else.

RIGA 27:

La porta viene indicata come chiusa.

RIGA 29:

Il socket viene chiuso al termine di ogni scansione.

Questa operazione è importante perché:

- libera memoria;
- evita connessioni inutilmente aperte;
- migliora la stabilità del programma.

### 5.3.2 Risultati del test

I risultati derivati dal programma Python sono i seguenti:

```
┌──(simone㉿simone)-[~]
└─$ /usr/bin/python /home/simone/Documenti/scansione_Porte.py
==================================================
Questo programma controlla quali porte
sono APERTE o CHIUSE su un host.
==================================================
Inserire un indirizzo ip: 10.1.100.10
Scansione porte su 10.1.100.10...

[CHIUSA] Porta 20
[APERTA] Porta 21
[APERTA] Porta 22
[APERTA] Porta 23
[APERTA] Porta 25
[APERTA] Porta 53
[APERTA] Porta 80
[CHIUSA] Porta 110
[CHIUSA] Porta 143
[APERTA] Porta 445
[APERTA] Porta 3306
[CHIUSA] Porta 3389
[APERTA] Porta 5432
[APERTA] Porta 5900
[CHIUSA] Porta 8080
[CHIUSA] Porta 8443
[CHIUSA] Porta 27017
```

### 5.3.3 Analisi

La scansione rivela che le porte aperte sono:

21, 22, 23, 25, 53, 80, 443, 445, 3306, 5432, 5900

Mentre le porte chiuse:

20, 110, 143, 8080, 8443, 27017

In questo caso, quello che è rilevante nell'analisi sono principalmente i servizi esposti verso la rete e le possibili implicazioni dal punto di vista della sicurezza informatica. La presenza di numerose porte aperte evidenzia infatti una superficie di attacco piuttosto ampia.

### 5.3.4 Consigli di sicurezza, TELNET, FTP, http

- **Telnet porta 23**

Tra gli elementi più significativi emersi dalla scansione vi è sicuramente la presenza della **porta 23** associata al protocollo **Telnet**. Questo tipo di protocollo rappresenta una criticità importante poiché trasmette dati e credenziali in chiaro, senza alcun meccanismo di cifratura.

**La squadra** predisposta all'analisi di sicurezza **consiglia all'azienda Theta di disabilitare il servizio Telnet e sostituirlo con protocolli più sicuri, come SSH**, in grado di garantire la cifratura del traffico di rete e delle credenziali di autenticazione.

- **FTP porta 21**

Anche **la presenza della porta 21** relativa al **servizio FTP** può rappresentare un **rischio**.

FTP, infatti, non utilizza cifratura per il trasferimento dei dati e potrebbe esporre credenziali sensibili qualora configurato in modo non sicuro. **Si consiglia di sostituirlo con protocolli più sicuri, come SFTP o FTPS**, che garantiscono la cifratura delle comunicazioni e del trasferimento dei file.

- **Http porta 80**

Per quanto riguarda la porta 80, **la scansione ha confermato la presenza di un servizio web HTTP attivo** sulla macchina Metasploitable 2. La presenza di un web server esposto in rete rappresenta un elemento particolarmente delicato dal punto di vista della sicurezza, poiché costituisce uno dei **principali punti di accesso verso il sistema**. Eventuali vulnerabilità presenti nelle applicazioni web o nella configurazione del server **potrebbero infatti essere sfruttate da un attaccante per ottenere informazioni sensibili o compromettere il sistema.** Si raccomanda caldamente di **sostituire HTTP con HTTPS,** in modo da garantire la cifratura delle comunicazioni tra client e server e proteggere eventuali dati sensibili trasmessi in rete.

In conclusione, per quanto riguarda anche le altre porte riscontrate come aperte nella scansione, come consiglio generale il team incaricato dell'analisi di sicurezza suggerisce all'azienda Theta di adottare una politica di rafforzamento dell'infrastruttura di rete andando ad implementare, aggiornare e sostituire eventuali protocolli non più sicuri.

# 6 Conclusioni

Per quanto riguarda le conclusioni è volontà del team Arancinə esprimere alcune considerazioni riguardo il progetto e l'attività di gruppo così come impostata dalla traccia. Il progetto per l'azienda Theta si è sviluppato sull'arco di tre/quattro giorni lavorativi ed ha interessato tutte le aree di interesse delle settimane precedenti riguardo la configurazione delle reti, l'analisi dei servizi esposti, la sicurezza informatica e lo sviluppo di strumenti in linguaggio Python. L'attività ha permesso al team di applicare concretamente le competenze teoriche affrontate durante il percorso formativo, simulando uno scenario realistico di analisi e testing di una infrastruttura di rete. L'attività di gruppo è stata anch'essa formativa, il team Arancinə composto da Michele, Riccardo, Simone Sal., Simone San. ed Andrea ha collaborato in maniera coordinata durante tutte le fasi del progetto, suddividendo i compiti in base alle competenze e contribuendo attivamente sia alla parte pratica che alla documentazione finale. Inoltre, tale attività ci ha permesso di legare anche umanamente Questa attività ci ha permesso anche di legare umanamente, rafforzando la collaborazione e il rapporto tra i membri del team attraverso il lavoro condiviso e il confronto continuo durante lo sviluppo del progetto. L'esperienza si è rivelata quindi non soltanto utile dal punto di vista tecnico e formativo, ma anche importante sotto il profilo umano e organizzativo.